Exercise: Product Discount System

A shop calculates the final price of a product using these discount rules:

- Price below 100 DKK: no discount
- Price from 100 to 499 DKK: 10% discount
- Price of 500 DKK or more: 20% discount

Part 1 — Given C# class

Create Product.cs in the Codebase project:

```
namespace Codebase
{
    public class Product
    {
        public string Name { get; set; }
        public decimal Price { get; set; }

        public Product(string name, decimal price)
        {
            Name = name;
            Price = price;
        }

        public decimal CalculateFinalPrice()
        {
            if (Price >= 500)
                return Price * 0.80m;
            else if (Price >= 100)
                return Price * 0.90m;

            return Price;
        }
    }
}
```

Part 2 — Three Complete Manual Test Cases

TC-01 — Product with No Discount

Precondition: The Product class is implemented, and CalculateFinalPrice() is available.

Steps:

1. Create a Product object: new Product("Book", 80).

2. Call the CalculateFinalPrice() method.

3. Compare the actual result with the expected result.

Expected result: 80 DKK (no discount applied).

Actual result: Record after execution.

Status: Not Run / Pass / Fail

TC-02 — Product with 10% Discount

Precondition: The Product class is implemented, and CalculateFinalPrice() is available.

Steps:

1. Create a Product object: new Product("Keyboard", 200).

2. Call the CalculateFinalPrice() method.

3. Verify that a 10% discount is applied.

4. Compare the actual result with the expected result.

Expected result: 180 DKK (200 − 20 = 180).

Actual result: Record after execution.

Status: Not Run / Pass / Fail

TC-03 — Product with 20% Discount

Precondition: The Product class is implemented, and CalculateFinalPrice() is available.

Steps:

1. Create a Product object: new Product("Monitor", 600).

2. Call the CalculateFinalPrice() method.

3. Verify that a 20% discount is applied.

4. Compare the actual result with the expected result.

Expected result: 480 DKK (600 − 120 = 480).

Actual result: Record after execution.

Status: Not Run / Pass / Fail

Part 3 — Write NUnit Unit Tests

Create ProductTests.cs in UnitTests. Complete the TODO sections using Arrange–Act–Assert.

```
using Codebase;
using NUnit.Framework;

namespace UnitTests
{
    [TestFixture]
    public class ProductTests
    {
        // TC-01: No discount
        [Test]
        public void Price80_NoDiscount()
        {
            // TODO: Create Product object
            // TODO: Calculate final price
            // TODO: Assert expected result
        }

        // TC-02: 10% discount
        [Test]
        public void Price200_TenPercentDiscount()
        {
            // TODO: Write test
        }

        // TC-03: 20% discount
        [Test]
        public void Price600_TwentyPercentDiscount()
        {
            // TODO: Write test
        }
    }
}
```

Part 4 — Run the Tests

In Zed, open the terminal in the solution folder and run:

```
dotnet test UnitTests/UnitTests.csproj
```

Expected after completing the tests: Total 3, Passed 3, Failed 0. Make sure UnitTests references Codebase and has NUnit installed.